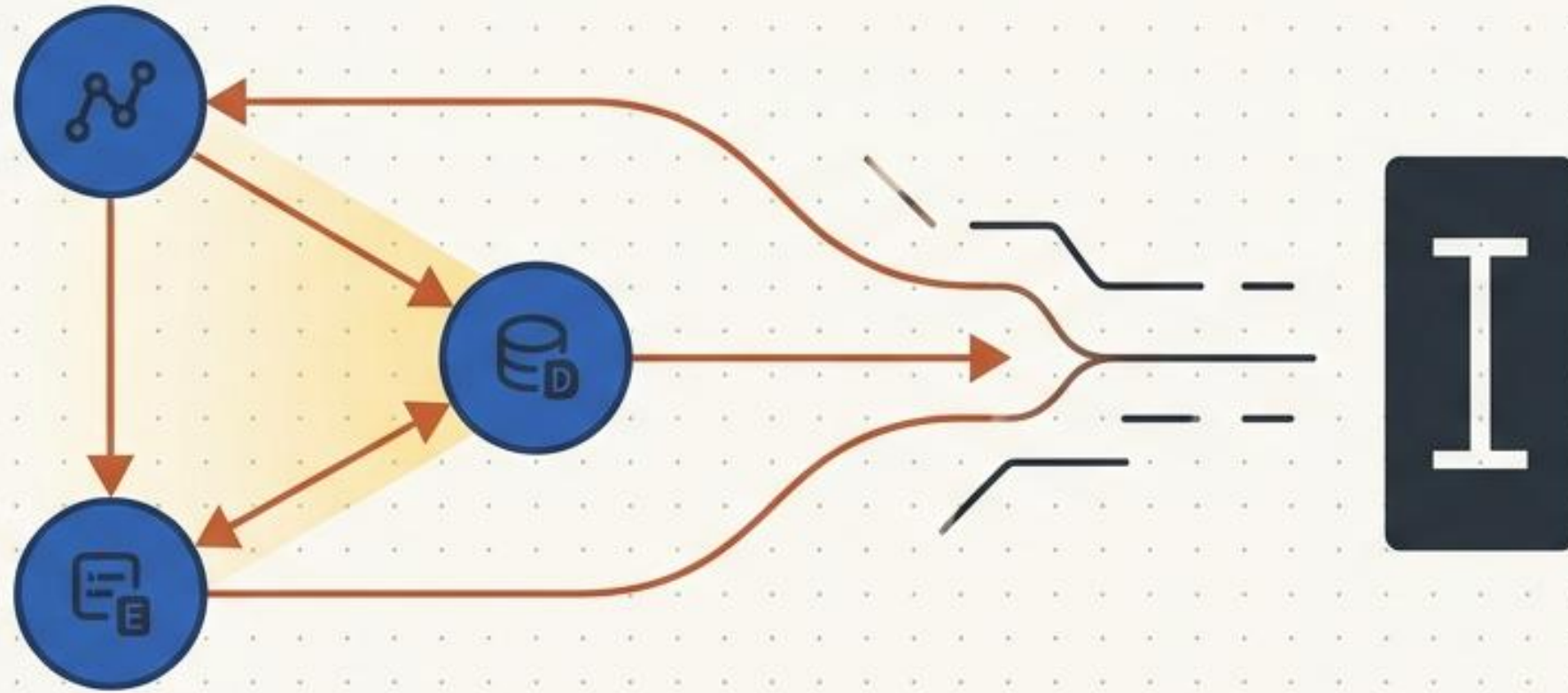


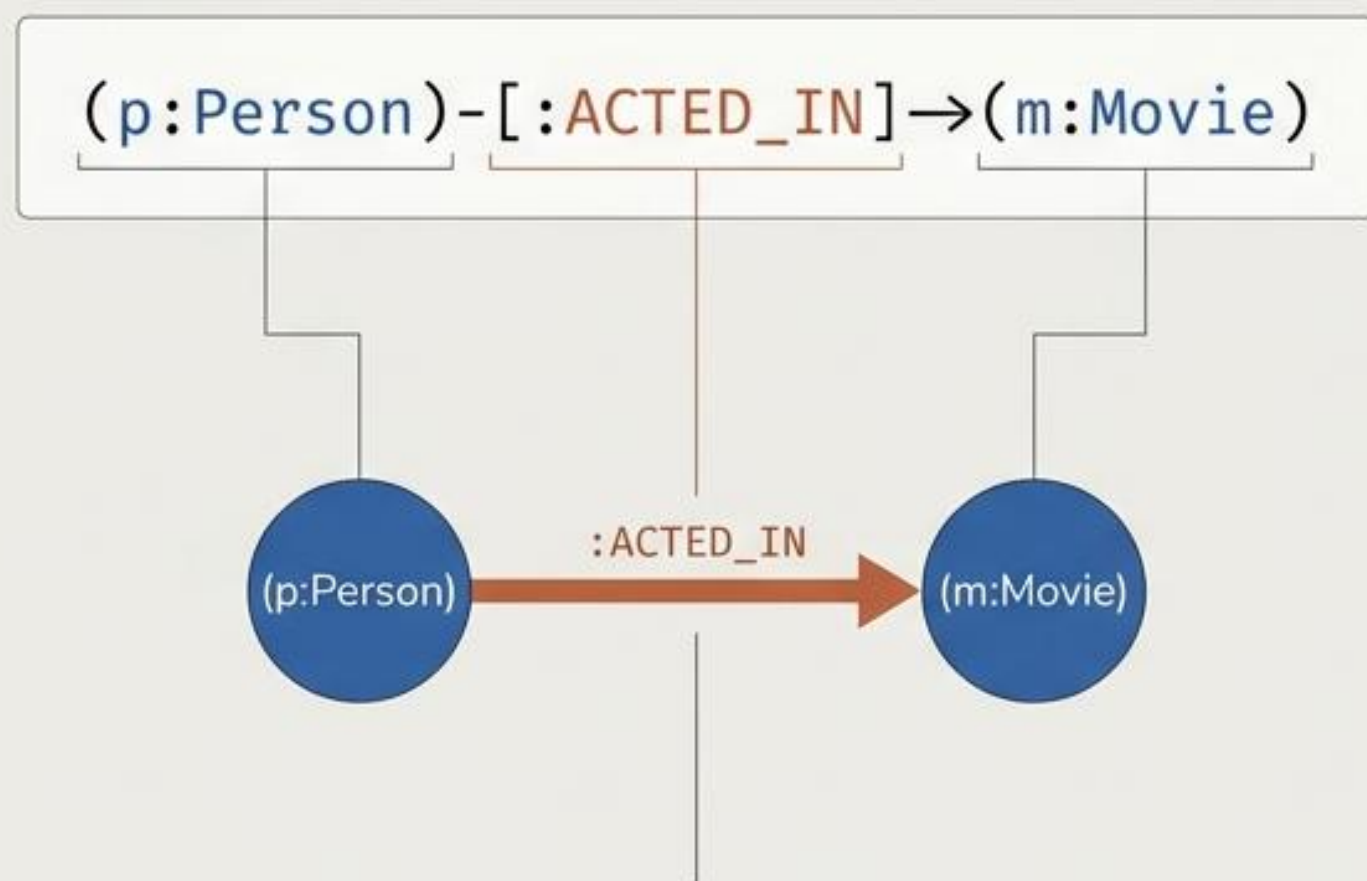
Cypher Basics - GQL for Neo4j



dr Zbigniew Galar

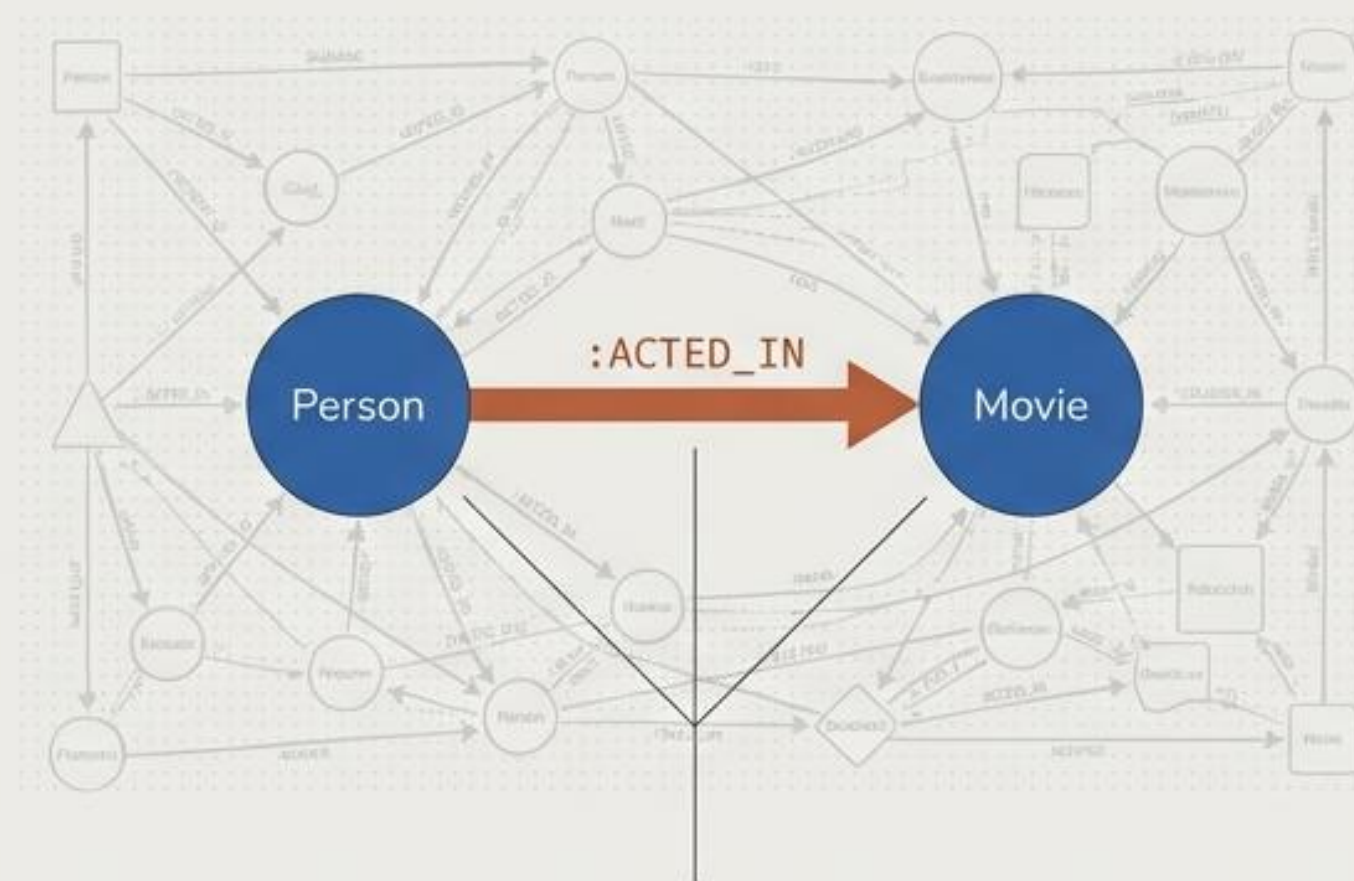
A Declarative Graph Query Language

The “What” (Cypher)



You describe the pattern.

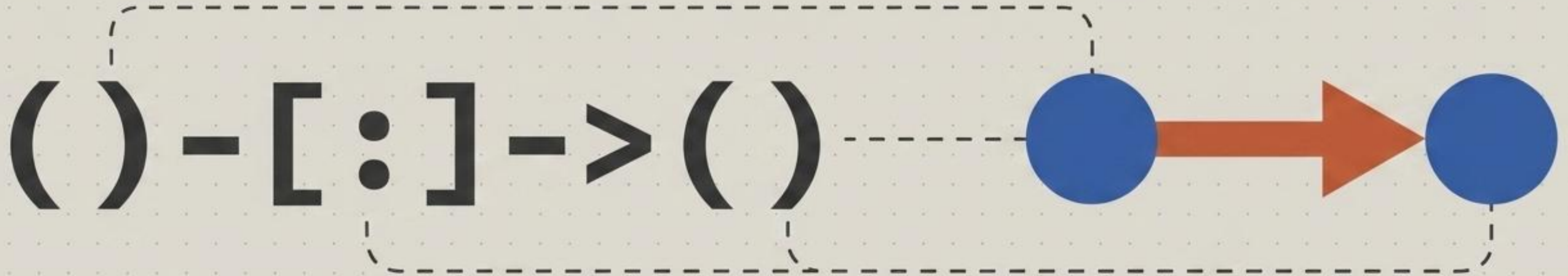
The “How” (Database Engine)



The database finds the optimal path.

The ASCII-Art Metaphor

Identify patterns using intuitive syntax consisting of brackets, dashes, and arrows.



Anatomy of a Pattern: Nodes

Node Canvas:

Expressed with parentheses to represent a circular node.

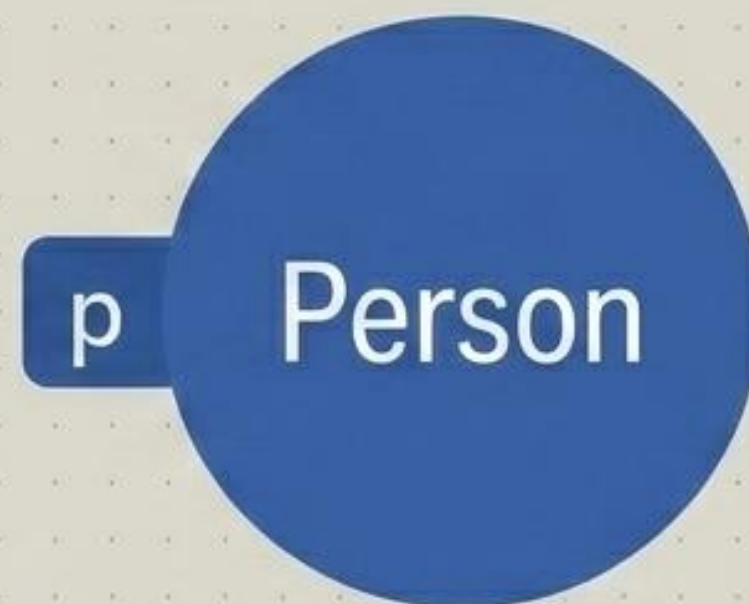
Variable / Alias:

Positioned before the node info to reference this specific entity later.

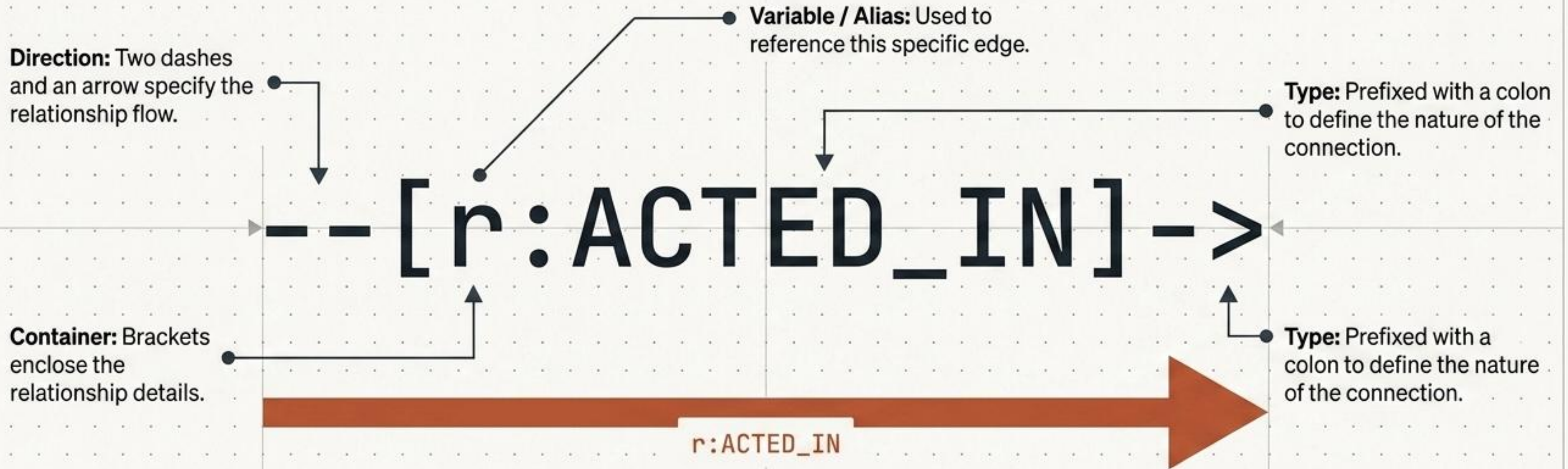
Label:

Prefixed with a colon to define the node's overarching category.

(p:Person)



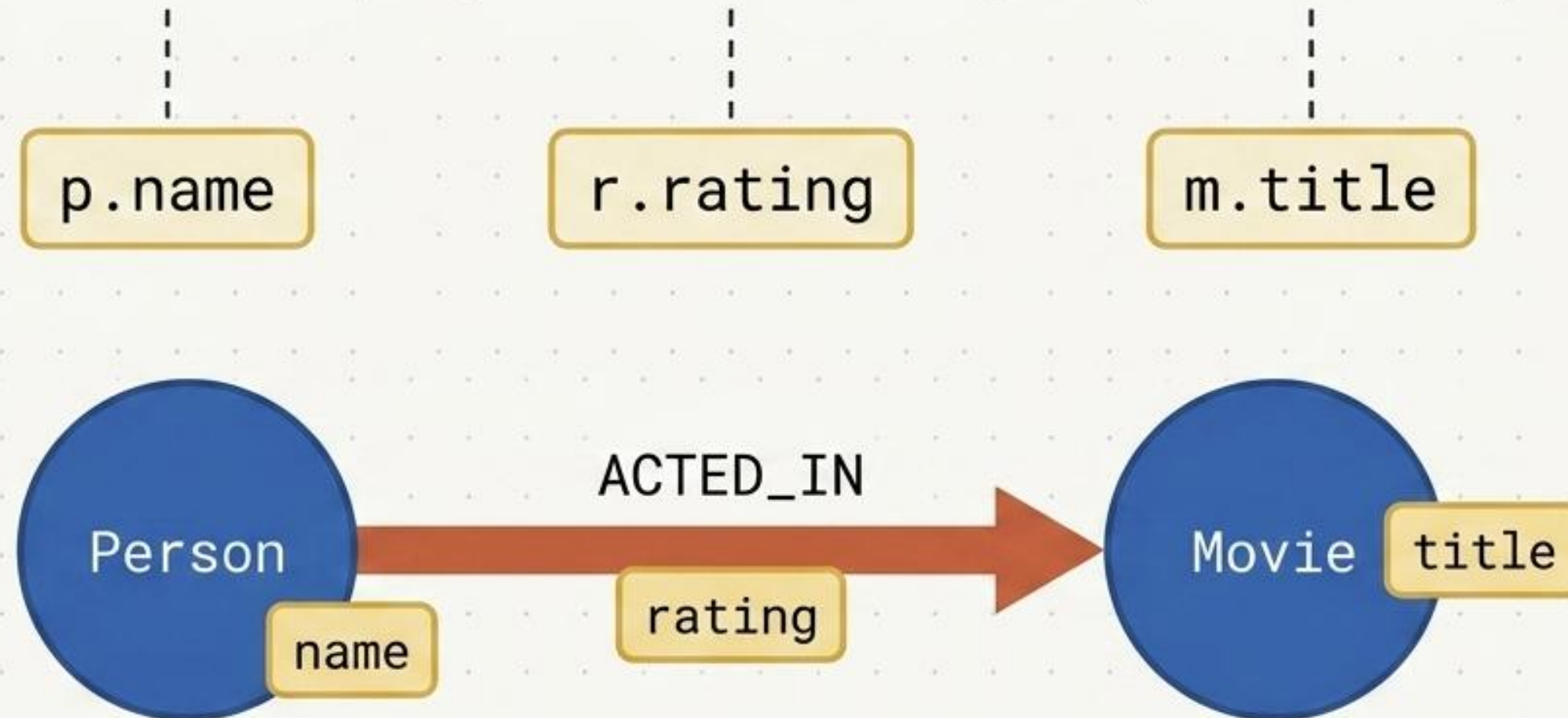
Anatomy of a Pattern: Relationships



Anatomy of a Pattern: Accessing Properties

Variables map the visual structure to accessible internal data attributes using **dot notation**.

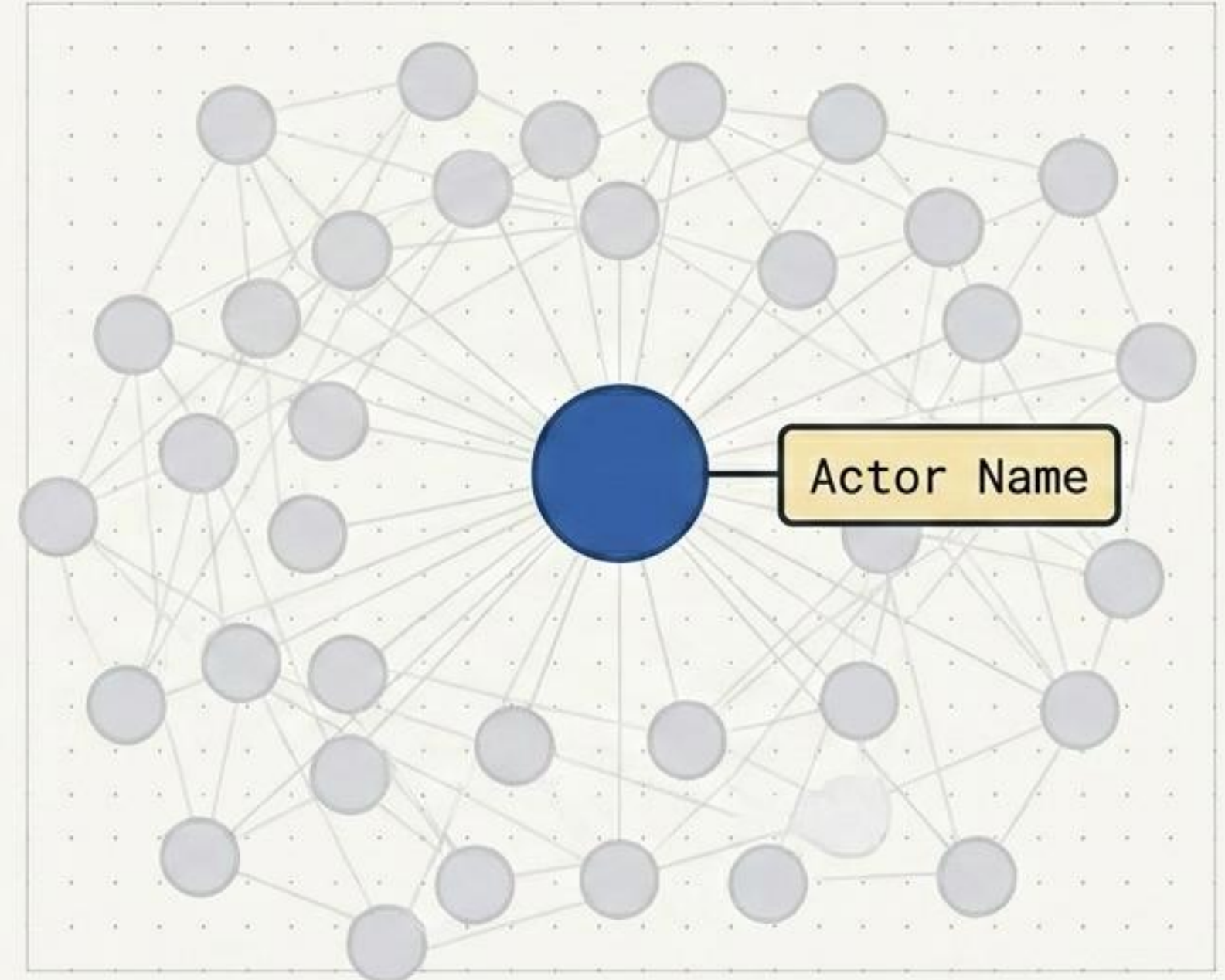
$(p:Person) - [r:ACTED_IN] \rightarrow (m:Movie)$



Finding Nodes with the MATCH Clause

The database is searched by defining the exact pattern you want to retrieve.

```
MATCH (n:Person)  
WHERE n.name = 'Actor Name'  
RETURN n
```

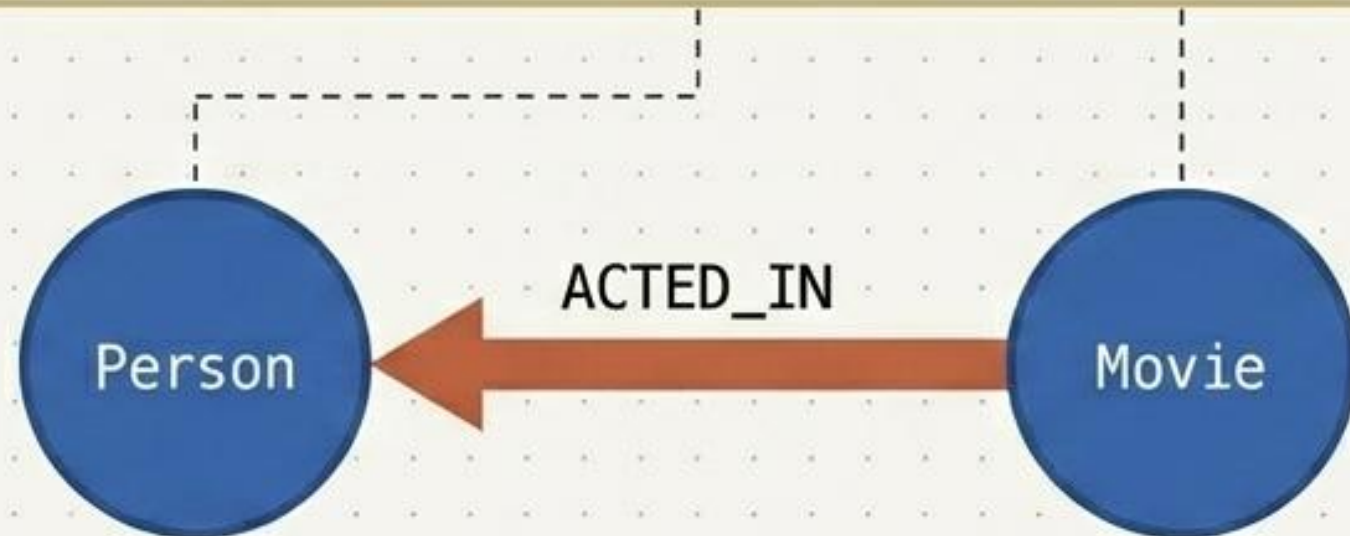


Traversing Relationships in the Graph

MATCH can navigate any connected domain simply by changing the labels and directions.

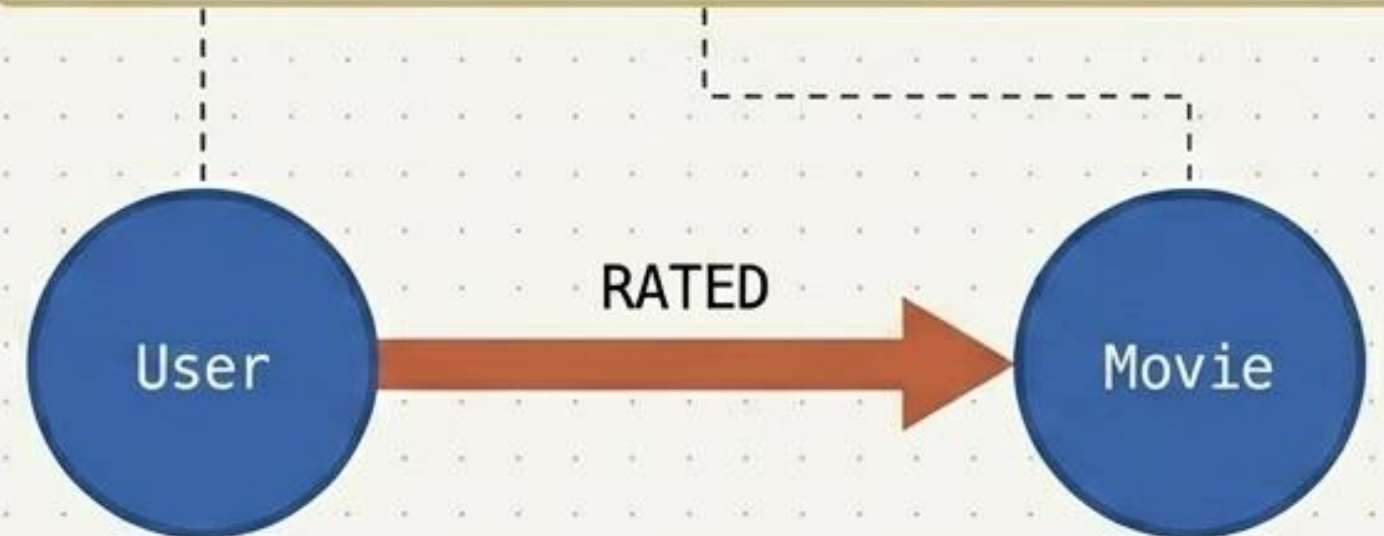
Movies Domain

```
MATCH (m:Movie)<-[r:ACTED_IN]-(p:Person)
WHERE m.title = 'Movie Title'
RETURN m, r, p
```



Users Domain

```
MATCH (u:User)-[r:RATED]->(m:Movie)
WHERE u.name = 'Mr. Jason Love'
RETURN u, r, m
```



Executing Multi-Hop Traversals

Chain multiple relationships together in a single query.

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(p2:Person)
WHERE p.name = 'Actor Name'
RETURN p2.name AS actor, m.title AS movie, r.role AS role
```



Formatting Output with the RETURN Clause

Transforming graph structures into structured tabular data.

THE INPUT

```
MATCH (u:User)-[r:RATED]->(m:Movie)
RETURN u.name, r.rating, m.title
```



RETURN Clause

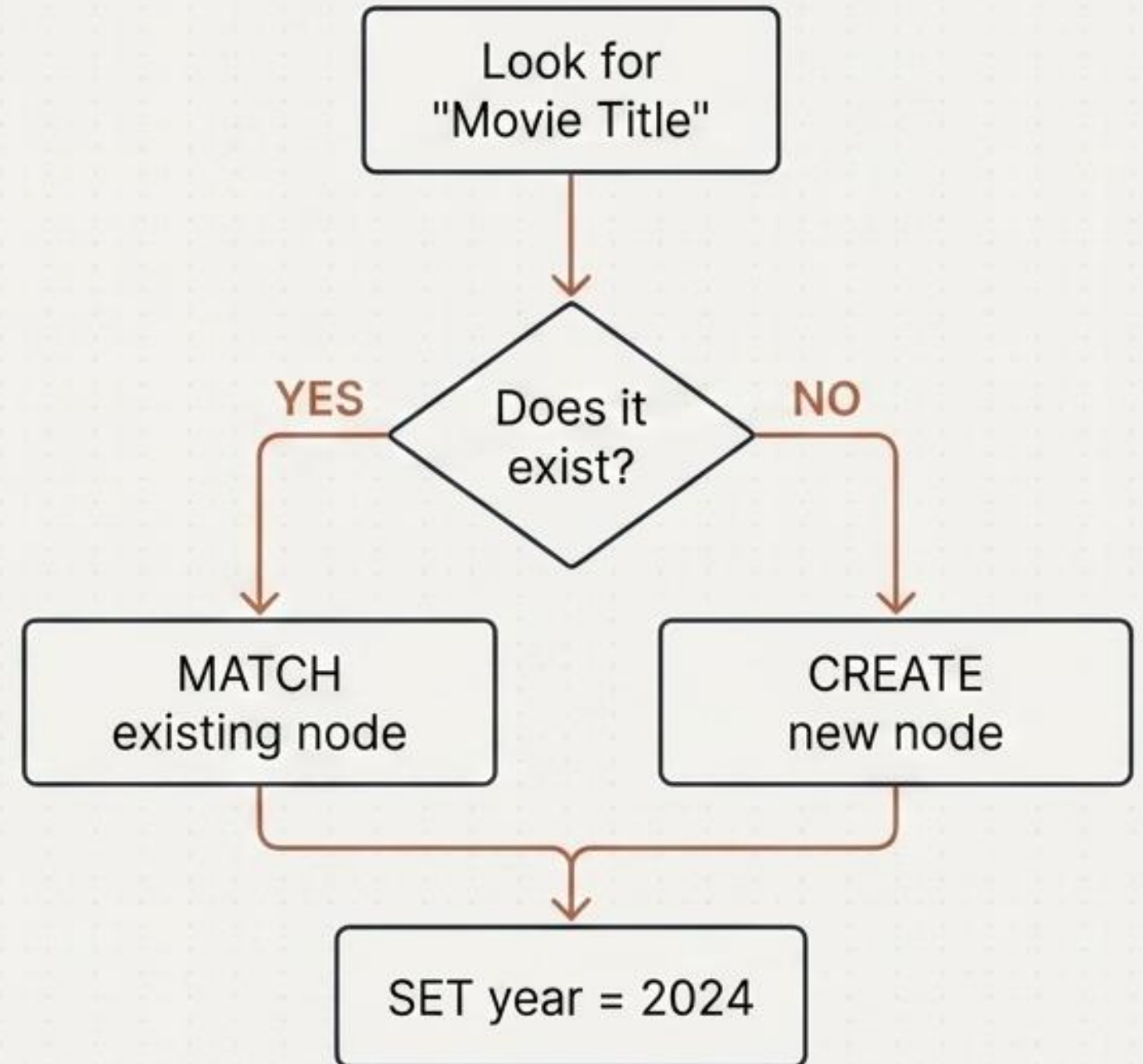
THE OUTPUT

u.name	r.rating	m.title

Modifying the Graph with the MERGE Clause

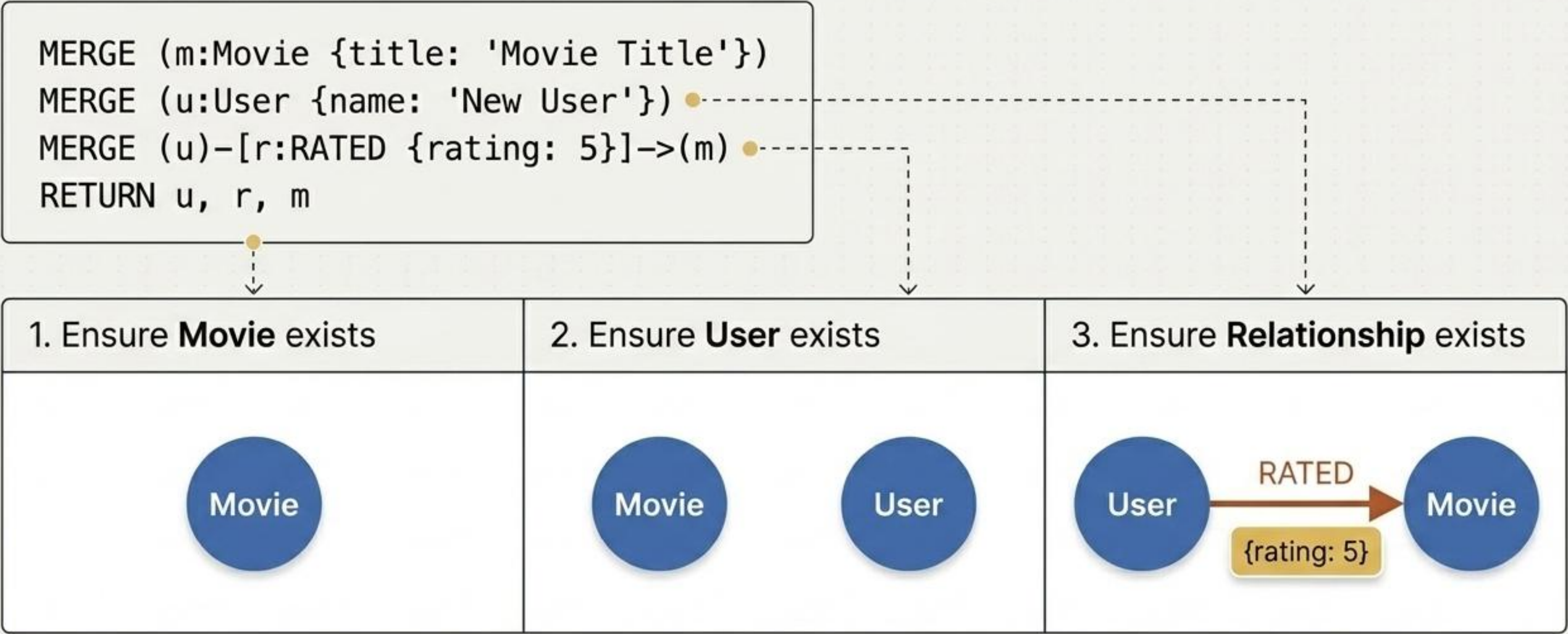
MERGE ensures a pattern exists by combining the functionality of MATCH and CREATE.

```
MERGE (m:Movie {title: 'Movie Title'})  
SET m.year = 2024  
RETURN m
```



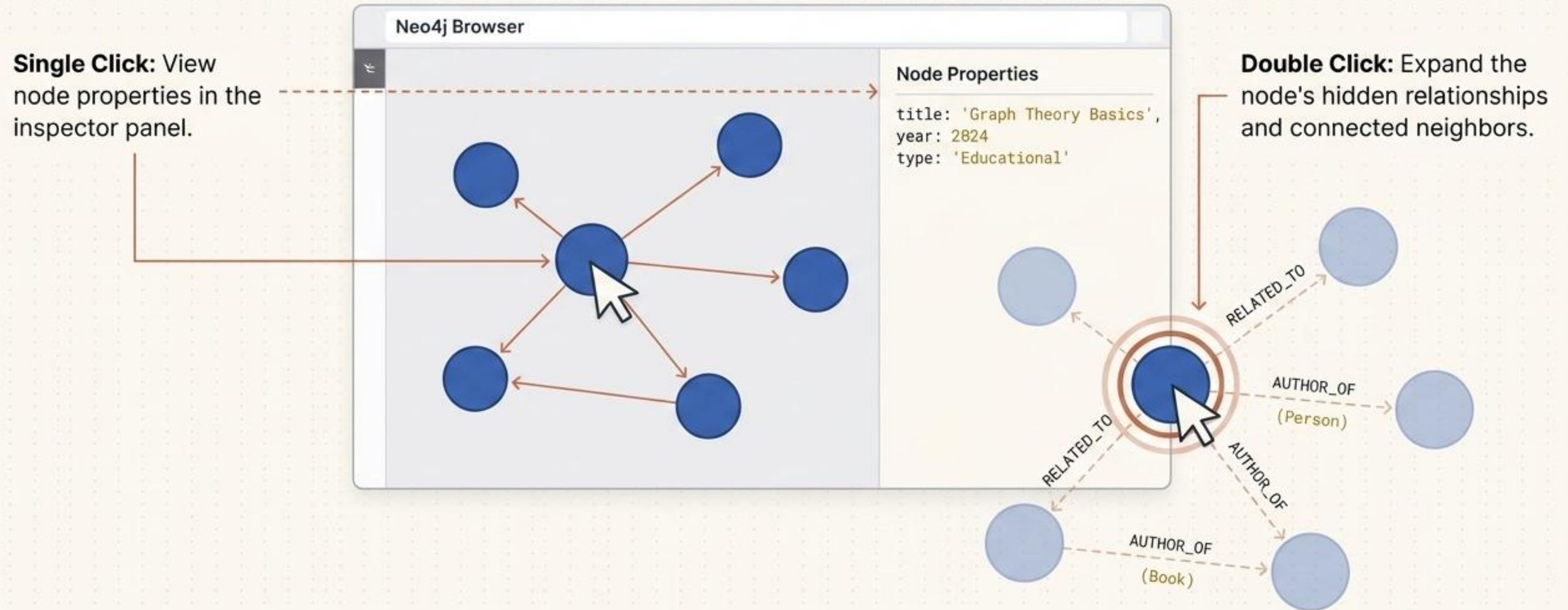
Constructing Complex Graph Structures

Chaining MERGE statements to build out relationships without duplicating data.



Interacting with the Neo4j Browser

The visual interface allows for intuitive, click-based exploration of query results.



Synthesis: Core Cypher Clauses

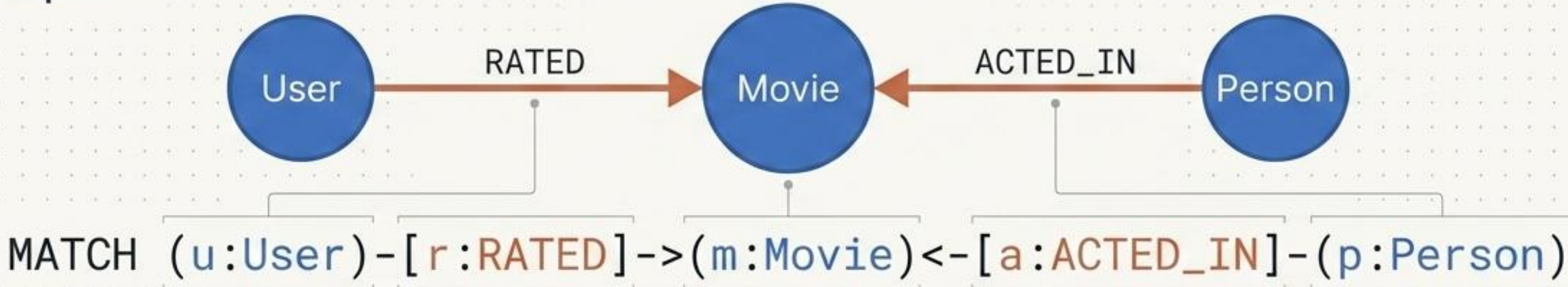
Clause	Function	Syntax Pattern	SQL Equivalent
MATCH	Find Patterns / Read	MATCH (n)-[r]->(m)	SELECT ... JOIN
WHERE	Filter Data	WHERE n.property = X	WHERE
RETURN	Format Output	RETURN n.prop AS alias	SELECT ... AS
MERGE	Upsert / Write	MERGE (n) SET n.prop = X	INSERT ... ON CONFLICT

Cypher Visual Grammar: Cheat Sheet

The Key



The Blueprint



The Flow

